INDUSTRIAL TRAINING REPORT

NEWSLENS AI: A SMART REAL-TIME NEWS ANALYSER WITH AI-POWERED SUMMARISATION, SENTIMENT ANALYSIS, FAKE NEWS DETECTION AND MULTILINGUAL AUDIO

Submitted in partial fulfilment of the

Requirements for the award of

Degree of Bachelor of Technology in Computer Science & Engineering

Session: 2026–27

Submitted By:

Name: Zinat Nisa

University Roll No.: 24ESKCS782

Semester/Branch: V Semester / Computer Science & Engineering

SUBMITTED TO:

Department of Computer Science & Engineering

SWAMI KESHVANAND INSTITUTE OF TECHNOLOGY,
MANAGEMENT & GRAMOTHAN, JAIPUR

DECLARATION

I hereby declare that the Industrial Training Report entitled "NewsLens AI: A Smart Real-Time News Analyser with AI-Powered Summarisation, Sentiment Analysis, Fake News Detection and Multilingual Audio" is an authentic record of my own work as requirements of the compulsory 45-days Industrial Training during the period from 15/05/2026 to 06/07/2026 at Amigzo Technologies Private Limited, Delhi for the award of degree of B.Tech. (Computer Science & Engineering), Swami Keshvanand Institute of Technology, Management & Gramothan, Jaipur, under the guidance of Mr. Vikash Dhankar (Assistant Professor, Department of Computer Science & Engineering).

_____

(Signature of Student)
Zinat Nisa
University Roll No.: 24ESKCS782
Semester/Branch: V Sem / CSE
Date:

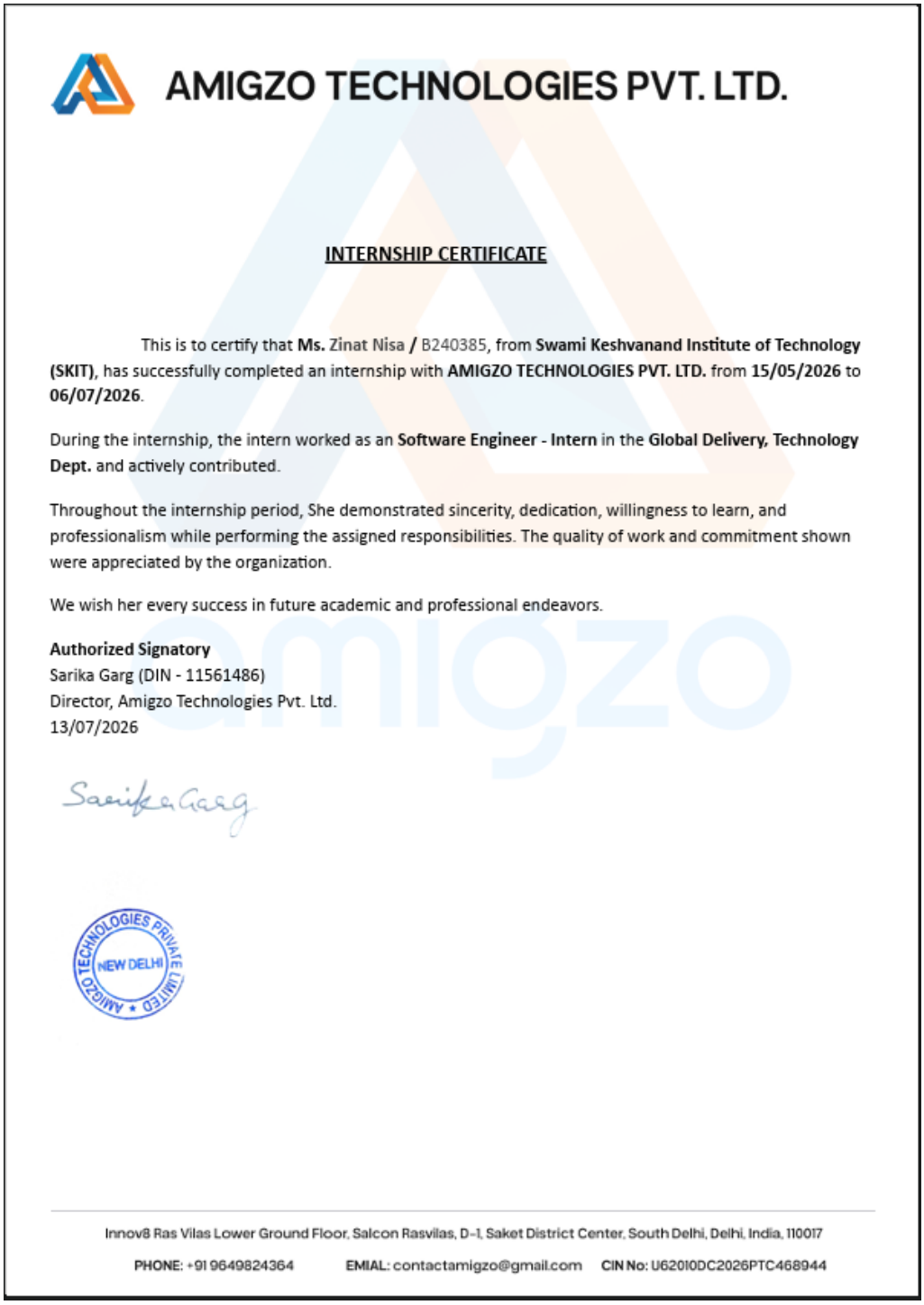
Certified that the above statement made by the student is correct to the best of our knowledge and belief.

Examined by:

Mr. Vikash Dhankar
Assistant Professor
Department of Computer Science & Engineering

Dr. Mehul Mahrishi
Head of Department
Department of Computer Science & Engineering

CERTIFICATE OF INTERNSHIP



AMIGZO TECHNOLOGIES PVT. LTD.

INTERNSHIP CERTIFICATE

This is to certify that **Ms.** Zinat Nisa / B240385, from **Swami Keshvanand Institute of Technology (SKIT)**, has successfully completed an internship with **AMIGZO TECHNOLOGIES PVT. LTD.** from **15/05/2026** to **06/07/2026**.

During the internship, the intern worked as an **Software Engineer - Intern** in the **Global Delivery, Technology Dept.** and actively contributed.

Throughout the internship period, She demonstrated sincerity, dedication, willingness to learn, and professionalism while performing the assigned responsibilities. The quality of work and commitment shown were appreciated by the organization.

We wish her every success in future academic and professional endeavors.

Authorized Signatory
Sarika Garg (DIN - 11561486)
Director, Amigzo Technologies Pvt. Ltd.
13/07/2026

Sarika Garg

AMIGZO TECHNOLOGIES PRIVATE LIMITED NEW DELHI

Innov8 Ras Vilas Lower Ground Floor, Salcon Rasvilas, D-1, Saket District Center, South Delhi, Delhi, India, 110017

PHONE: +91 9649824364 EMIAL: contactamigzo@gmail.com CIN No: U62010DC2026PTC468944

ACKNOWLEDGEMENT

First and foremost, I wish to express my sincere thanks and deepest gratitude to my esteemed Industry Mentors at Amigzo Technologies Private Limited and Ms. Sarika Garg (Director), who contributed immensely to the successful completion of my Industrial Training through their insightful mentorship, regular technical reviews, and valuable guidance in Python development, API integration, and applied Artificial Intelligence.

I would also like to express my sincere gratitude to my Faculty Facilitator, Mr. Vikash Dhankar (Assistant Professor, Department of Computer Science & Engineering), for his continuous guidance, valuable feedback, and scholarly supervision throughout the tenure of this industrial training.

I tender my sincere thanks to Dr. Mehul Mahrishi, Head of the Department, Computer Science & Engineering, for providing the necessary departmental infrastructure, constant encouragement, and academic support.

I am equally grateful to Mr. Deepak Goyal, Mr. Vikram Khandelwal and Mr. Shivank Lavania, Coordinators (III Year Internship 2026–27), for their administrative support, cooperation, and motivation provided during the industrial training.

Finally, I extend my heartfelt thanks to my parents, family, and peers whose encouragement and unwavering belief enabled me to complete this project and report with dedication.

Zinat Nisa
University Roll No.: 24ESKCS782
B.Tech. V Semester (CSE)

ABSTRACT

The volume of news published online every day has grown far beyond what any individual reader can consume. Readers face information overload, difficulty in judging which reports are trustworthy, a lack of time to read complete articles, and a language barrier that excludes many people from news published in a language they do not read comfortably. Conventional news websites and aggregators deliver headlines but leave the work of understanding, verifying and translating them entirely to the reader.

To address these challenges, this Industrial Training project presents **NewsLens AI**—a smart, real-time news analyser delivered as a single web application. NewsLens AI lets a user search any topic or choose a trending one, fetches the latest live headlines through RSS feeds, and then applies Large Language Model (LLM) analysis to each selected report. For every analysed item the system produces a short, key-point summary, a positive/neutral/negative sentiment classification presented as a chart, and an AI-generated credibility review that flags claims which appear unreliable. The summary can be translated into a language chosen by the user and is also converted into speech so that the news can be listened to as well as read.

The application is implemented entirely in Python. The user interface is built with Streamlit, live headlines are parsed with the `feedparser` library, natural-language understanding tasks are performed through the OpenAI API, text-to-speech is produced with `gTTS`, and sentiment charts are drawn with `matplotlib`. The application is deployed on Streamlit Community Cloud and is publicly reachable through a web link. The project demonstrates how a compact, one-pipeline architecture—*User Input* $\rightarrow$ *RSS Feed* $\rightarrow$ *OpenAI API* $\rightarrow$ *UI* + *Audio*—can combine five capabilities (live news, AI summary, sentiment analysis, fake news detection, and translation with audio) in one simple, accessible platform.

Keywords: Real-Time News Analysis, Natural Language Processing, Large Language Models, OpenAI API, Streamlit, RSS, Sentiment Analysis, Fake News Detection, Text-to-Speech, Machine Translation.

Table of Contents

1 Introduction **1**

1.1 Background and Motivation 1

1.2 About the Organisation 2

1.3 Project Description 2

1.4 Project Scope 2

1.5 Intended Users 3

1.6 Core Project Features 3

2 Tools & Technology Used **5**

2.1 Software Development Lifecycle (SDLC) Model 5

2.2 System Architecture 6

2.3 Technology Stack Overview 6

2.4 Data Model and Processing Design 7

2.5 Design and Implementation Constraints 8

3 Snapshots and Implementation **9**

3.1 Proposed System Workflow and Data Flow 9

3.1.1 News Discovery and Search Workflow 9

3.1.2 AI Analysis Workflow 9

3.1.3 Credibility Review Workflow 10

3.2 Project Interface Snapshots 10

3.2.1 Home – Live News Feed 10

3.2.2 AI Summary, Translation and Audio 11

3.2.3 Sentiment Analysis 12

3.2.4 Fake News Detection 12

3.3 Key Source Code Implementation 13
3.3.1 Live Headline Retrieval with RSS 13
3.3.2 LLM-Based Summary, Sentiment and Credibility Analysis 13
3.3.3 Translation, Text-to-Speech and Sentiment Chart 14

4 Results and Discussion **15**
4.1 System Verification and Functional Testing 15
4.2 Evaluation of the AI Analysis Features 16
4.2.1 Summarisation Quality 16
4.2.2 Sentiment Analysis 16
4.2.3 Credibility Review 16
4.2.4 Accessibility through Translation and Audio 16
4.3 Security and Privacy Considerations 16
4.4 Comparative Analysis with Existing Solutions 17

5 Future Scope and Limitations **18**
5.1 Project Conclusion and Summary 18
5.2 Key Technical Learnings and Takeaways 18
5.3 Current Limitations 19
5.4 Future Scope and Engineering Roadmap 19

6 Conclusion **21**

Bibliography **22**

Chapter 1

Introduction

1.1 Background and Motivation

Over the past decade, the way people consume news has moved almost completely from printed newspapers and scheduled television bulletins to continuously updated online platforms. News portals, social media feeds, and mobile applications publish thousands of reports every hour. Although this gives readers unprecedented access, it also introduces several critical challenges:

1. **Information Overload:** A huge amount of news is published every single day. Even on a single topic such as a sporting event or an election, a reader may be shown dozens of overlapping reports from different outlets, making it hard to find the essential facts.
2. **Difficulty in Judging Trust and Relevance:** Misleading headlines, unverified claims and deliberately fabricated stories circulate alongside genuine journalism. It is difficult for an ordinary reader to quickly identify which news is relevant and trustworthy.
3. **Lack of Time:** Most users do not have time to read every full article. They want the key points quickly, along with some idea of the overall tone of the coverage.
4. **Language Barrier:** A large proportion of online news is published in English, which limits accessibility for readers who are more comfortable in other languages, and for readers who prefer to listen rather than read.

Recent advances in Large Language Models (LLMs) make it possible to condense text, classify its sentiment, assess the plausibility of claims, and translate it into other languages with a single general-purpose model. Combined with freely available RSS news feeds and text-to-speech tools, these capabilities can be packaged into a lightweight application that directly addresses the problems above. This observation motivated the development of NewsLens AI.

1.2 About the Organisation

Amigzo (registered as Amigzo Technologies Private Limited) is an Indian career-technology start-up based in Delhi. It is a mentorship platform that connects people one-on-one with working industry professionals, based on the belief that direct guidance from the right person can make learning more focused and practical. Instead of generic courses or scattered online forums, users get direct access to mentors (called "Guides") who have real, current industry experience. Guides set their own hours, topics and per-minute rates, while sessions are kept private.

During the Industrial Training at Amigzo, the author worked as a Software Engineer–Intern in the Global Delivery, Technology Department and developed NewsLens AI, gaining hands-on experience in Python, AI/NLP, API integration, Streamlit and full-stack development in a professional environment.

1.3 Project Description

NewsLens AI is an AI-powered, real-time news analyser delivered as a web application. It fetches live headlines through RSS feeds and, on demand, creates AI summaries and sentiment analysis, performs a fake-news review, and translates the result into a chosen language with an audio version—all inside one simple interface.

Unlike a plain news reader, NewsLens AI treats a headline as the starting point rather than the end product. Each fetched item can be sent through an analysis pipeline built on the OpenAI API, and the outcome is presented to the user as readable key points, a sentiment chart, a credibility explanation, translated text, and an audio player. The application is built with Python, Streamlit and the OpenAI API and is publicly accessible as a hosted web app.

1.4 Project Scope

The scope of the NewsLens AI project spans data acquisition, applied artificial intelligence, user interface development and cloud deployment carried out during the industrial training:

- **Live Data Acquisition:** Retrieval of current headlines for any user-entered topic or predefined trending topic using RSS feeds parsed with `feedparser`.
- **AI Summarisation:** Condensing long news content into short, easy-to-read key points using the OpenAI API.
- **Sentiment Analysis and Visualisation:** Classifying the tone of news as positive, neutral or negative and displaying the distribution as a chart drawn with `matplotlib`.

- **Fake News Detection:** Prompting the language model to review the reported claims and explain whether they look credible or unreliable.
- **Multi-Language Translation and Text-to-Speech:** Translating the generated output into the language selected by the user and producing an audio version with `gTTS`.
- **User Interface:** A single-page Streamlit interface with a settings sidebar, topic search, trending-topic shortcuts, per-article analysis buttons and a search history.
- **Deployment:** Publishing the application on Streamlit Community Cloud so that it is available through a public URL.

1.5 Intended Users

NewsLens AI is structured to serve a wide spectrum of news consumers:

1. **Students and Competitive-Exam Aspirants:** Users who must keep track of current affairs daily and benefit from short key-point summaries instead of full articles.
2. **Working Professionals:** Readers with limited time who want a quick, reliable understanding of the day's developments in areas such as the stock market and start-ups.
3. **Regional-Language and Non-Native English Readers:** Users who prefer to read news in their own language through the translation feature.
4. **Visually Impaired and On-the-Move Users:** Users who benefit from listening to the news through the text-to-speech audio player.
5. **General Readers Concerned About Misinformation:** Users who want an additional AI-assisted credibility check before trusting or sharing a report.

1.6 Core Project Features

The primary features engineered into NewsLens AI include:

- **Live News and Topic Search:** Search any news topic and fetch live headlines through RSS feeds. Trending-topic shortcuts (AI, Cricket, Elections, Startup, and Stock Market) provide one-click access to popular subjects.
- **AI Summarisation:** Long articles are condensed into short, numbered key points that can be read in seconds.

- **Sentiment Analysis with Chart:** Positive, neutral and negative sentiment of the analysed items is shown in a clear bar chart.
- **Fake News Detection:** The AI reviews the news content and flags claims that look unreliable, explaining the reasoning behind its assessment.
- **Multi-Language Translation and Text-to-Speech:** The user can read the news in the language of their choice from the settings sidebar and listen to the audio version.
- **Per-Article Analysis:** Each headline card carries its own "Analyze" button, so analysis is performed only for the items the user is interested in.
- **Search History:** Previously searched topics are listed in the sidebar for quick re-use.

Chapter 2

Tools & Technology Used

2.1 Software Development Lifecycle (SDLC) Model

The development of NewsLens AI followed the Agile Iterative Model. Because the project combines several loosely coupled capabilities—data retrieval, language-model prompting, audio generation and user interface design—the iterative approach allowed each capability to be built, tested against real news and refined independently before being integrated into the final pipeline.

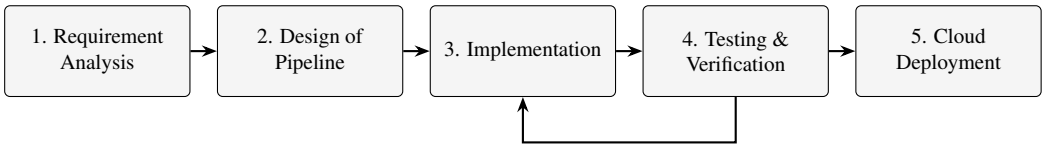


Figure 2.1: Iterative SDLC Workflow Applied in NewsLens AI Development

The project progressed across four iterations:

1. **Iteration 1 (Data Acquisition & Interface Skeleton):** Setting up the Streamlit application, the settings sidebar, topic search, and RSS retrieval with `feedparser` to display live headline cards.

2. **Iteration 2 (AI Analysis Core):** Integrating the OpenAI API and designing the prompts for summarisation and sentiment classification.

3. **Iteration 3 (Credibility, Translation & Audio):** Adding fake-news review, multi-language translation, `gTTS` audio generation and the `matplotlib` sentiment chart.

4. **Iteration 4 (Usability & Deployment):** Adding trending topics, search history and per-article analysis controls, followed by testing and deployment on Streamlit Community Cloud.

2.2 System Architecture

NewsLens AI implements a compact, pipeline-oriented client–server architecture in which a Python application hosted in the cloud orchestrates calls to external services and renders the results in the browser:

- **Presentation Tier (Streamlit Frontend):** A Streamlit web interface running in the browser. It provides the settings sidebar (language selector, topic search, trending topics, search history) and the main panel that shows headline cards, summaries, translations, audio player, charts and the credibility review.
- **Application Tier (Python Logic):** Python functions that fetch and parse RSS feeds, build prompts, call the OpenAI API, convert text to speech, and generate charts. Streamlit's session state keeps the current topic, the analysed article and the search history between interactions.
- **External Services Tier:** The RSS news source that supplies live headlines, the OpenAI API that performs summarisation, sentiment analysis, fake-news review and translation, and the Google Text-to-Speech service accessed through `gTTS`.

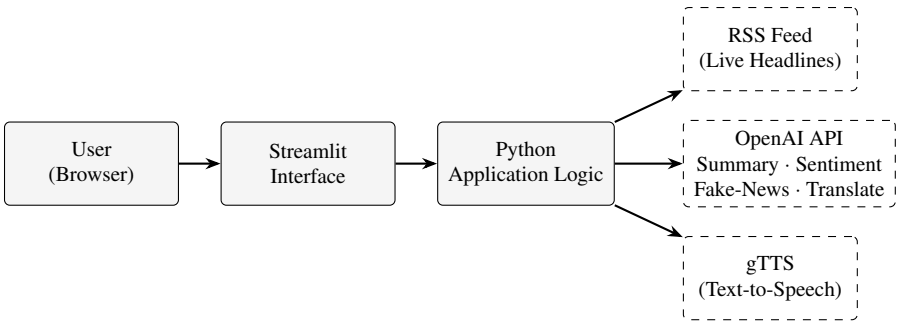


Figure 2.2: High-Level Architecture of NewsLens AI

2.3 Technology Stack Overview

Table 2.1 details the technology stack used in the engineering and deployment of NewsLens AI.

Layer	Technology	Purpose and Functional Role
Frontend Framework	Streamlit	Builds the interactive web interface (sidebar, cards, buttons, audio player, charts) directly from Python.
Backend Language	Python 3	Core application logic, data handling and orchestration of all services.
AI / NLP Engine	OpenAI API	LLM used for summarisation, sentiment classification, fake-news review and translation.
Data Source	RSS Feeds	Supplies live news headlines, source names and article links for any searched topic.
Feed Parsing	feedparser	Parses RSS/Atom feeds into structured Python objects.
Text-to-Speech	gTTS	Converts the translated summary into an audio file played in the browser.
Visualisation	matplotlib	Draws the positive/neutral/negative sentiment bar chart.
Session Management	Streamlit session state	Retains search history, selected topic and analysed article across reruns.
Cloud Hosting	Streamlit Community Cloud	Hosts the application and exposes it through a public URL.
Version Control	Git & GitHub	Source-code management and deployment source for the hosted application.

Table 2.1: Comprehensive Technology Stack of NewsLens AI

2.4 Data Model and Processing Design

NewsLens AI is deliberately stateless with respect to persistent storage: no database is required, because the news itself is fetched live on every request. The application instead works with a small set of in-memory entities that flow through the pipeline:

1. **Topic Query:** The text entered in the search box or chosen from the trending topic buttons. It forms the search term sent to the RSS source and is appended to the search history in session state.

2. **News Item:** One entry parsed from the RSS feed, comprising the headline title, the originating source (publisher), and the link to the full article.

3. **Analysis Request:** The text of the selected news item(s) together with the prompt instructions sent to the OpenAI API for one of the tasks (summary, sentiment, credibility).

4. **Analysis Result:** The structured text returned by the model: numbered key points, one sentiment label per point, and a free-text credibility explanation.

5. **Localised Output:** The result translated into the language code chosen in the sidebar, and the corresponding generated audio file.



Figure 2.3: Data Entities Flowing Through the NewsLens AI Pipeline

2.5 Design and Implementation Constraints

During development and deployment, several technical constraints were encountered and engineered around:

- **Streamlit Rerun Model:** Streamlit re-executes the whole script on every user interaction. Expensive operations such as API calls therefore had to be triggered only by explicit button presses (the per-article "Analyze" buttons) and their outputs preserved in session state to avoid repeated cost and latency.
- **API Cost and Latency:** Every LLM call has a monetary cost and a response time. Analysis is therefore performed on demand for selected items and the prompts request concise, bounded output.
- **Secure Handling of Credentials:** The OpenAI API key must never be committed to a public repository. The key is supplied to the application through environment/secret configuration rather than hard-coded in the source.
- **Free-Tier Hosting:** Deployment on Streamlit Community Cloud means the app runs in a shared, resource-limited container, which favours a lightweight design without a persistent database or heavy local models.
- **Language Coverage of Text-to-Speech:** Audio is only available for languages supported by the `gTTS` service, so the language selector is tied to the codes that the service accepts.

Chapter 3

Snapshots and Implementation

3.1 Proposed System Workflow and Data Flow

NewsLens AI's operational logic follows one simple, real-time pipeline: *User Input* $\rightarrow$ *RSS Feed* $\rightarrow$ *OpenAI API* $\rightarrow$ *UI + Audio*.

3.1.1 News Discovery and Search Workflow

1. The user types a topic into the *Search Topic* box or clicks one of the trending topic buttons in the sidebar.
2. The application builds an RSS request for the topic and retrieves the latest headlines using `feedparser`.
3. Each entry is parsed into a title, publisher name and article link.
4. The interface renders each item as a card with a "Read more" link and an "Analyze" button, and records the topic in the search history.

3.1.2 AI Analysis Workflow

Once a user presses an analysis button, the following sequence is executed:

1. **Search:** The user selects a topic or a trending topic.
2. **Fetch:** RSS feeds bring in the latest live headlines.
3. **Analyse:** The OpenAI API is prompted to produce a numbered summary, a sentiment label for each point, and a fake-news review of the reported claims.
4. **Translate:** The summary is converted to the language chosen in the sidebar.

5. **Show:** The UI displays the summary, translation, sentiment chart and credibility review, together with an audio player generated by `gTTS`.

3.1.3 Credibility Review Workflow

1. The headlines and available content for the selected topic are passed to the language model with an instruction to act as a fact-checking assistant.

2. The model examines whether the claims are consistent with the type of reporting expected from legitimate news outlets, whether named people, scores or records are plausible, and whether any sensational or manipulative elements are present.

3. The model returns an explanation, including any caveat that full verification against real-time events may still be required, which is displayed under the *Fake News Detection* heading.

3.2 Project Interface Snapshots

This section presents the primary user interfaces of the deployed application, which is available at https://news-lens-ai-zbk2q2qxny8bjknwxzd9dt.streamlit.app.

3.2.1 Home – Live News Feed

Figure 3.1 shows the home screen. The left sidebar contains the *Settings* panel with the translation language selector, the topic search box, the *Trending Topics* shortcuts (AI, Cricket, Elections, Startup, Stock Market) and the *Search History*. The main panel lists live headlines as cards, each with a "Read more" link and an "Analyze" button.

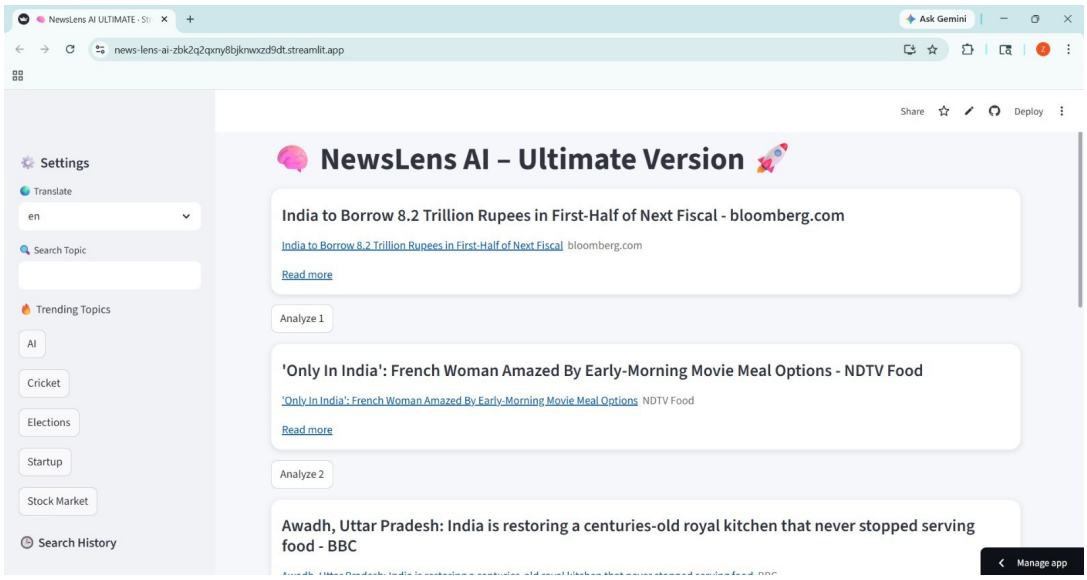


Figure 3.1: NewsLens AI Home Screen: Live News Feed with Settings Sidebar

3.2.2 AI Summary, Translation and Audio

Figure 3.2 shows the output for the topic “ipl”. The AI condenses the news into three numbered key points, repeats them in the language selected in the sidebar under the *Translation* heading, and provides an embedded audio player for the text-to-speech version of the translated text.

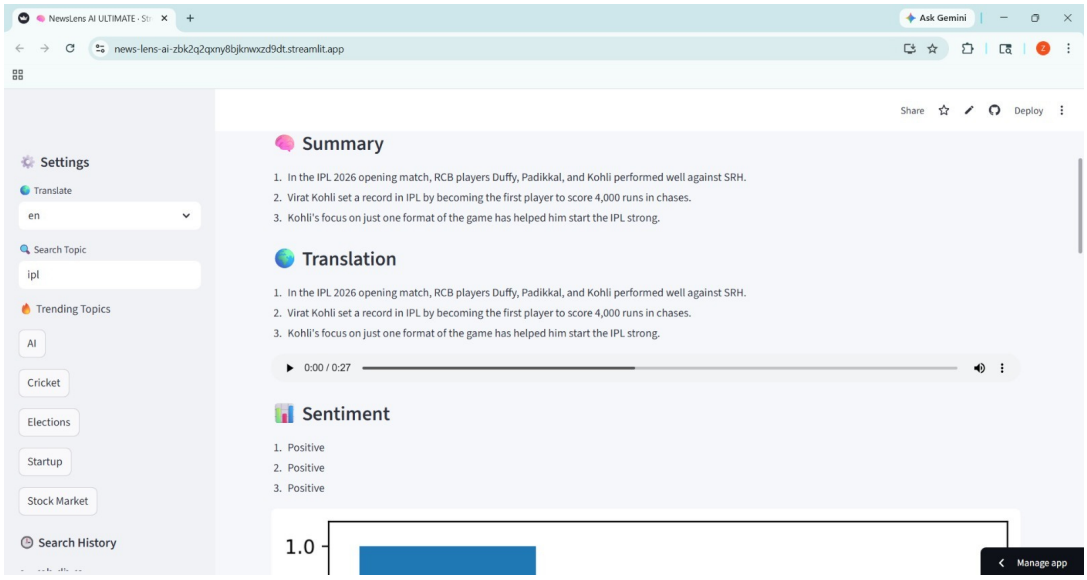


Figure 3.2: AI-Generated Summary, Translation and Text-to-Speech Audio Player

3.2.3 Sentiment Analysis

Figure 3.3 illustrates the sentiment section. Each key point is labelled positive, neutral or negative, and the overall distribution is plotted as a bar chart with *Positive*, *Neutral* and *Negative* categories.

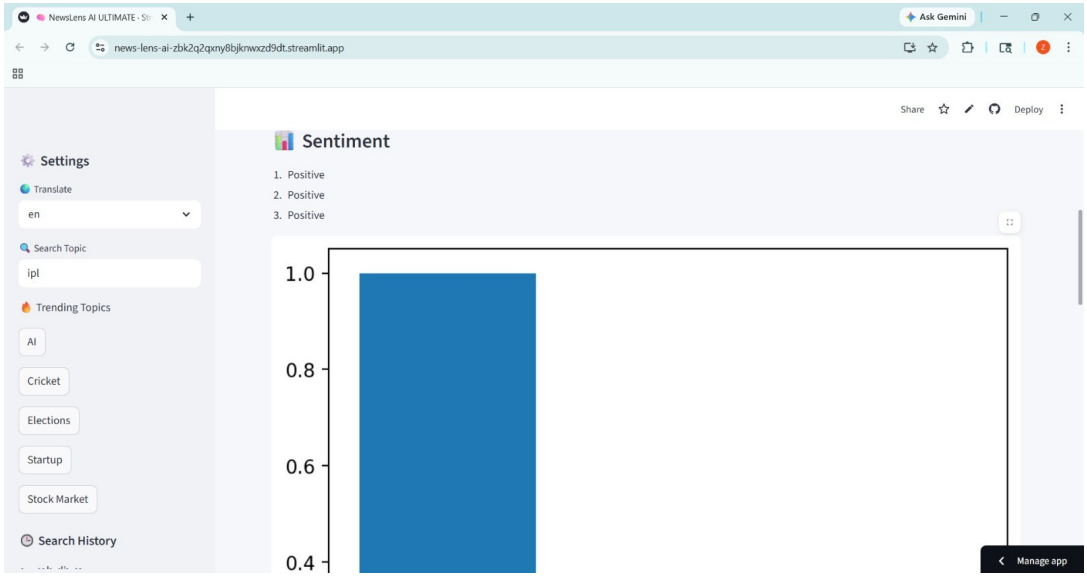


Figure 3.3: Sentiment Analysis of the News Presented as a Chart

3.2.4 Fake News Detection

Figure 3.4 displays the *Fake News Detection* section, in which the AI explains in plain language whether the reported claims look credible, followed by further headline cards that can be analysed in the same way.

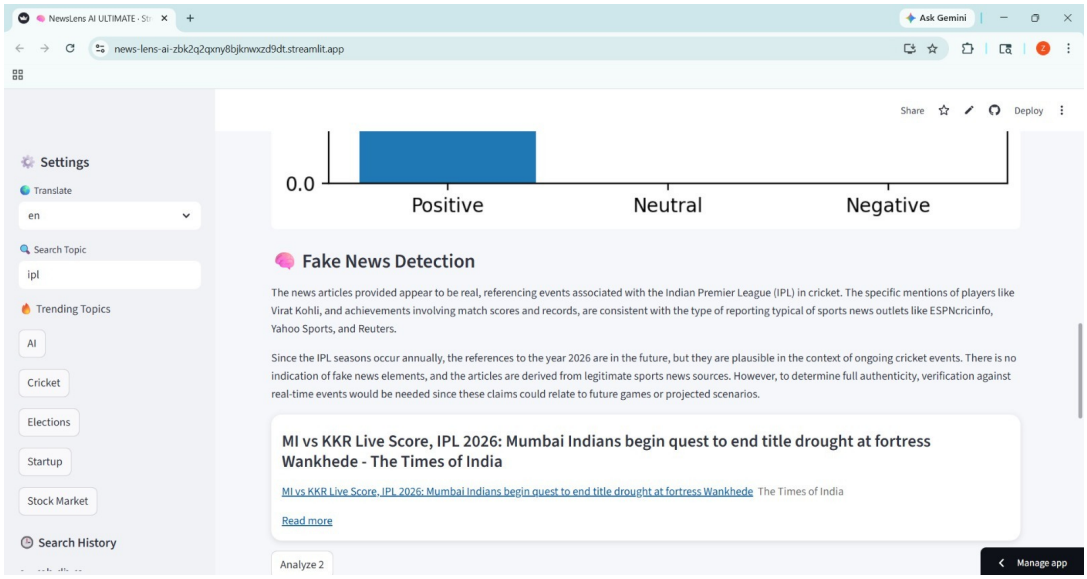


Figure 3.4: AI Credibility Review under the Fake News Detection Section

3.3 Key Source Code Implementation

This section highlights the critical code segments of the NewsLens AI pipeline.

3.3.1 Live Headline Retrieval with RSS

Listing 3.1 shows the function that converts a topic into an RSS request and returns the parsed headline entries.

```
# app.py
import feedparser
from urllib.parse import quote_plus

def fetch_news(topic: str, limit: int = 10):
    query = quote_plus(topic) if topic else ""
    url = (f"https://news.google.com/rss/search?q={query}"
           "&hl=en-IN&gl=IN&ceid=IN:en") if topic else \
          "https://news.google.com/rss?hl=en-IN&gl=IN&ceid=IN:en"
    feed = feedparser.parse(url)
    return [
        {
            "title": e.title,
            "link": e.link,
            "source": e.get("source", {}).get("title", ""),
        }
        for e in feed.entries[:limit]
    ]
```

Listing 3.1: Fetching live headlines using feedparser

3.3.2 LLM-Based Summary, Sentiment and Credibility Analysis

Listing 3.2 demonstrates how a single helper function sends task-specific prompts to the OpenAI API to obtain the summary, the sentiment labels and the fake-news review.

```
# ai_utils.py
from openai import OpenAI
client = OpenAI()          # API key read from environment / secrets
MODEL = "gpt-4o-mini"

def ask(prompt: str) -> str:
    resp = client.chat.completions.create(
        model=MODEL,
        messages=[{"role": "user", "content": prompt}],
        temperature=0.3,
```

```
    )
    return resp.choices[0].message.content.strip()

def summarise(text):
    return ask("Summarise the following news in 3 short numbered "
               f"key points:\n{text}")

def sentiment(points):
    return ask("For each numbered point, answer only Positive, "
               f"Neutral or Negative, one per line:\n{points}")

def fake_news_check(text):
    return ask("Act as a fact-checking assistant. Review these news "
               "items and explain whether the claims look credible "
               f"or unreliable:\n{text}")
```

Listing 3.2: Prompting the OpenAI API for the three analysis tasks

3.3.3 Translation, Text-to-Speech and Sentiment Chart

Listing 3.3 presents the output stage, which translates the summary, generates the audio and draws the sentiment chart inside the Streamlit interface.

```
# app.py (output stage)
import io, streamlit as st
import matplotlib.pyplot as plt
from gtts import gTTS

lang = st.sidebar.selectbox("Translate", ["en", "hi", "ta", "bn"])

translated = ask(f"Translate to language code '{lang}':\n{summary}")
st.subheader("Translation")
st.write(translated)

audio = io.BytesIO()
gTTS(text=translated, lang=lang).write_to_fp(audio)
st.audio(audio.getvalue(), format="audio/mp3")

labels = [s.strip() for s in sentiment_text.splitlines() if s.strip()]
counts = [labels.count(c) / len(labels)
          for c in ("Positive", "Neutral", "Negative")]
fig, ax = plt.subplots()
ax.bar(["Positive", "Neutral", "Negative"], counts)
st.pyplot(fig)
```

Listing 3.3: Translation, audio generation and sentiment chart

Chapter 4

Results and Discussion

4.1 System Verification and Functional Testing

The deployed application was verified through end-to-end testing with live topics. The tests exercised each feature of the pipeline using real RSS headlines, including the topic "ipl" used for the snapshots in Chapter 3. Table 4.1 summarises the functional test cases and the observed behaviour.

No.	Feature	Test Input	Observed Result	Status
1	Home news feed	Application launch	Live headline cards loaded with source name, "Read more" link and "Analyze" button.	Pass
2	Topic search	Topic text "ipl"	Headlines relevant to the topic were fetched and the topic was added to search history.	Pass
3	Trending topics	Click on AI / Cricket / Elections / Startup / Stock Market	One-click retrieval of live headlines for the chosen topic.	Pass
4	AI summarisation	Analysis of the "ipl" headlines	Three concise numbered key points covering the opening match performances and a batting record.	Pass
5	Translation	Language code selected in sidebar	Translated key points displayed under the Translation heading.	Pass
6	Text-to-speech	Translated summary	Audio player rendered with a playable clip of about 27 seconds.	Pass
7	Sentiment analysis	Three key points	Each point labelled; bar chart plotted for Positive, Neutral and Negative.	Pass
8	Fake news detection	Same headlines	Plain-language credibility explanation produced, with a caveat on verification.	Pass

Table 4.1: Functional Test Cases and Observed Results

4.2 Evaluation of the AI Analysis Features

4.2.1 Summarisation Quality

For the "ipl" topic, the model reduced several headline-level reports into three short statements: the performance of named players in the opening match, a batting record achieved in run chases, and a remark on the player's focus on one format of the game. The key points were readable at a glance, faithful to the reported topic, and short enough to be converted to audio of well under a minute.

4.2.2 Sentiment Analysis

Every key point in the test run was classified as *Positive*, which is consistent with the celebratory tone of the sports coverage. The chart presented this as a single dominant *Positive* bar with no neutral or negative counterpart, making the overall tone of the coverage immediately visible without reading the text.

4.2.3 Credibility Review

The fake-news module did not simply return a verdict; it explained its reasoning. For the test topic it noted that the mentioned players, scores and records were consistent with typical reporting from established sports news outlets and that no fake-news elements were indicated. It also added a caveat that full authenticity would require verification against real-time events. This behaviour is desirable because it presents the result as decision support rather than as an absolute ruling.

4.2.4 Accessibility through Translation and Audio

The combination of language selection and `gTTS` audio output allows the same summary to be consumed as text in a preferred language or as a spoken clip. The *Translate* selector in the settings sidebar keeps this option available throughout the session.

4.3 Security and Privacy Considerations

In line with good engineering practice, the following measures were applied:

1. **Credential Isolation:** The OpenAI API key is not hard-coded in the source files and is supplied through environment variables or the hosting platform's secret manager, so it is not exposed in the public repository.

2. **No Persistent Personal Data:** The application does not require user accounts and does not store personal information in a database. Search history is held only in the active session.

3. **Bounded Prompts and Outputs:** Prompts instruct the model to return concise, structured output, limiting both cost and the amount of unexpected content displayed.

4. **Source Attribution:** Every headline card shows the originating publisher and a link to the full article, so that users can always cross-check an AI result against the original source.

4.4 Comparative Analysis with Existing Solutions

Table 4.2 contrasts NewsLens AI with the broad categories of tools that readers commonly use today.

Feature / Capability	**News Websites**	**RSS Readers**	**Short-News Apps**	**NewsLens AI**
Live headlines	Yes	Yes	Yes	Yes
Topic search and trending shortcuts	Partial	Partial	Partial	Yes
AI key-point summary on demand	No	No	Partial	Yes
Sentiment analysis with chart	No	No	No	Yes
AI fake-news / credibility review	No	No	No	Yes
Translation to chosen language	Partial	No	Partial	Yes
Text-to-speech audio	Partial	No	Partial	Yes
All features in a single interface	No	No	No	Yes

Table 4.2: Comparative Analysis: NewsLens AI vs. Common News Consumption Tools

The comparison shows that NewsLens AI's strength lies in unifying five capabilities—live news, AI summary, sentiment analysis, fake-news detection, and translation with audio—in one platform, instead of requiring the reader to use several separate tools.

Chapter 5

Future Scope and Limitations

5.1 Project Conclusion and Summary

The Industrial Training carried out at Amigzo Technologies Private Limited provided an excellent platform to bridge theoretical computer science principles with professional software engineering practice. Through the conception, design, implementation and cloud deployment of NewsLens AI, a complete and publicly accessible AI-driven web application was delivered.

NewsLens AI directly addresses the problems of information overload, uncertain trust, lack of reading time and language barriers in online news. By combining live RSS retrieval with LLM-based summarisation, sentiment analysis, credibility review, translation and speech output, it turns the daily flood of headlines into quick, trustworthy and accessible insight.

5.2 Key Technical Learnings and Takeaways

The execution of this project fostered competencies across multiple areas:

1. **Python Application Development:** Structuring a multi-feature application in Python and building an interactive interface with Streamlit, including an understanding of its rerun and session-state model.

2. **Working with APIs and Feeds:** Consuming RSS feeds with `feedparser` and integrating a hosted language-model API into a real application.

3. **Applied NLP and Prompt Engineering:** Designing task-specific prompts for summarisation, sentiment classification, credibility assessment and translation, and handling the model's text responses reliably.

4. **Multimedia and Visualisation:** Producing audio with `gTTS` and charts with `matplotlib` and presenting them inside a web interface.

5. **Deployment and Credential Management:** Publishing an application through Streamlit Community Cloud and keeping API credentials out of source control.

6. **Professional Workflow:** Working with mentors in a start-up environment, taking regular feedback and delivering working increments within a fixed schedule.

5.3 Current Limitations

While NewsLens AI provides a functional platform, certain limitations exist in the current prototype:

- **Credibility Review is Model-Based:** The fake-news check relies on the language model's judgement of the headlines and its general knowledge; it does not cross-check claims against independent fact-checking databases. In the test run the model even treated references to the year 2026 as lying in the future, which shows that its knowledge may lag behind current events. Its verdict should therefore be treated as guidance, not proof.
- **Headline-Level Input:** RSS feeds supply titles, sources and links rather than full article bodies, so the analysis is largely based on headline-level information.
- **Dependence on External Services:** The application depends on the availability, rate limits and pricing of the OpenAI API, the RSS source and the `gTTS` service.
- **Limited Language Coverage for Audio:** Speech output is restricted to languages supported by `gTTS`, and the quality of translated audio depends on the translation.
- **Session-Only History:** Search history is not stored per user account across devices or sessions.
- **Coarse Sentiment Scale:** Sentiment is limited to three classes and does not give a confidence score.

5.4 Future Scope and Engineering Roadmap

To evolve NewsLens AI from a prototype into a comprehensive news-intelligence platform, the following enhancements are planned:

1. **Full-Article Extraction:** Fetching and analysing the complete text of each article rather than the headline alone for more accurate summaries.

2. **Evidence-Backed Fake-News Detection:** Combining the language model with retrieval from multiple trusted sources and fact-checking databases, and displaying a confidence score with citations.

3. **Source Reliability Scoring and Bias Indicators:** Rating publishers and showing how different outlets cover the same story.

4. **Personalisation and User Accounts:** Saved topics, bookmarks and persistent history synchronised across devices.

5. **Richer Analytics:** Sentiment trends over time, named-entity extraction and topic clustering for exploring a story's evolution.

6. **Voice and Mobile Experience:** Voice-based queries, a daily audio news briefing, and a mobile application or progressive web app.

7. **Regional-Language Expansion:** Wider support for Indian regional languages in both translation and speech output.

Chapter 6

Conclusion

The Industrial Training completed at Amigzo Technologies Private Limited has been an immensely enriching educational and professional experience. Undertaking the conceptualisation, design, implementation and deployment of NewsLens AI provided invaluable exposure to modern Python development, applied NLP and AI-assisted application design.

Throughout this project, several key milestones were successfully attained:

- **Real-Time News Delivery:** Built a live news feed with topic search and trending topics using RSS feeds, solving the first problem of access to current headlines.
- **AI Summarisation and Sentiment Insight:** Condensed news into short key points and visualised its overall tone through a sentiment chart, saving the reader's time.
- **Credibility Assistance:** Implemented an AI-based fake-news review that explains whether reported claims look reliable, helping users judge trust.
- **Inclusive Access:** Added multi-language translation and text-to-speech so that news can be read in the user's language and listened to as audio.
- **Public Deployment:** Deployed the complete application on Streamlit Community Cloud as a one-link, ready-to-use web app.

In conclusion, NewsLens AI makes news simpler to consume, faster to understand, smarter in its insights and more open to every reader. It demonstrates how a small, well-designed pipeline can combine live data with responsible use of generative AI to deliver one platform in which users can read, understand and listen to the news. The technical skills, discipline and problem-solving methods acquired during this training will serve as a strong cornerstone for future academic pursuits and professional engineering endeavours.

Bibliography

[1] Streamlit Inc., "Streamlit – A faster way to build and share data apps," Official Documentation, 2026. [Online]. Available: https://docs.streamlit.io

[2] OpenAI, "OpenAI API Documentation," 2026. [Online]. Available: https://platform.openai.com/docs

[3] Python Software Foundation, "Python Language Reference," 2026. [Online]. Available: https://docs.python.org/3/

[4] K. Kurkowski et al., "feedparser – Parse Atom and RSS feeds in Python," Documentation. [Online]. Available: https://feedparser.readthedocs.io

[5] P. N. Durette, "gTTS: Google Text-to-Speech Python library and CLI tool," Documentation. [Online]. Available: https://gtts.readthedocs.io

[6] J. D. Hunter, "Matplotlib: A 2D graphics environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.

[7] RSS Advisory Board, "RSS 2.0 Specification." [Online]. Available: https://www.rssboard.org/rss-specification

[8] T. Brown et al., "Language Models are Few-Shot Learners," in *Advances in Neural Information Processing Systems*, vol. 33, 2020.

[9] Streamlit Inc., "Streamlit Community Cloud – Deploy your app," Documentation, 2026. [Online]. Available: https://docs.streamlit.io/deploy/streamlit-community-cloud

[10] OWASP Foundation, "OWASP Top 10 Web Application Security Risks," 2023. [Online]. Available: https://owasp.org/Top10/